Understanding Python Decorators

Worked out solutions and explanations

Exercise 1 Solution: String Uppercase Decorator

Objective: *Write a basic decorator that converts the string output of a function to uppercase.*

```python
# Exercise 1: Basic Uppercase Decorator
# Task: Write a basic decorator that converts the string output of a function to uppercase.

from functools import wraps

def uppercase_decorator(func):
    """Converts the returned string value of the target function to uppercase."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        # Invoke the original function
        original_output = func(*args, **kwargs)

        # Validate that the output is indeed a string before calling .upper()
        if isinstance(original_output, str):
            return original_output.upper()
        return original_output
    return wrapper

# Sample usage:
@uppercase_decorator
def get_welcome_message(name):
    """Returns a personalized welcome message string."""
    return f"Welcome, {name}! Let's learn decorators."

if __name__ == "__main__":
    result = get_welcome_message("delegate")
    print(f"Result: {result}")
    print(f"Preserved Docstring: {get_welcome_message.__doc__}")
```

Explanation:

This solution demonstrates a basic outer-inner functional closure. The uppercase_decorator takes func as an argument. Inside, the wrapper capture closure executes the target, intercepts the returned value, checks its datatype, and returns the uppercase conversion. The @wraps decorator prevents original function namespace data loss.

Exercise 2 Solution: Double Invocation Decorator

Objective: *Create a decorator that executes the decorated target function twice on every call.*

```python
# Exercise 2: Double Invocation Decorator
# Task: Create a decorator that executes the decorated target function twice on every call.

from functools import wraps

def double_invocation(func):
    """Executes the target function twice, returning the result of the second execution."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("[DoubleInvocation] First execution start...")
        func(*args, **kwargs)  # First call (side-effects executed)
        print("[DoubleInvocation] Second execution start...")
        result = func(*args, **kwargs)  # Second call (capture output)
        return result
    return wrapper

# Sample usage:
@double_invocation
def send_alert(message):
    """Simulates sending an alert message."""
    print(f"ALERT: {message}")
    return "Alert Sent"

if __name__ == "__main__":
    status = send_alert("Database connection warning!")
    print(f"Final Status: {status}")
```

Explanation:

This solution modifies the target function's execution boundaries. Rather than altering parameters, the wrapper controls execution frequency. It triggers the original callable twice, capturing and returning the return value of only the second execution. This pattern is common in automatic reconnects and side-effect testing.

Exercise 3 Solution: Positional Argument Type Validator

Objective: *Write a decorator that validates that the first positional argument passed to a function is a string.*

```python
# Exercise 3: Type Validation Decorator
# Task: Write a decorator that validates the datatype of a function's arguments.

from functools import wraps

def type_check_string(func):
    """Validates that the first positional argument passed to the function is a string."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        if not args:
            raise ValueError("No positional arguments provided to type-check.")

        first_arg = args[0]
        if not isinstance(first_arg, str):
            raise TypeError(
                f"Type Validation Error: First argument must be of type 'str', "
                f"but received '{type(first_arg).__name__}'."
            )

        return func(*args, **kwargs)
    return wrapper

# Sample usage:
@type_check_string
def process_username(username):
    """Processes and registers a unique username."""
    print(f"Username '{username}' has been successfully processed.")
    return True

if __name__ == "__main__":
    try:
        process_username("alex_python")  # Success
        process_username(12345)          # Raises TypeError
    except TypeError as e:
        print(f"Caught expected error: {e}")
```

Explanation:

This guard pattern uses decorators to validate data pre-conditions before executing inner business logic. The wrapper intercepts the positional arguments tuple args. By checking args[0], it forces type validation, raising a TypeError early to prevent runtime issues deeper in the execution pipeline.

Exercise 4 Solution: Execution Logger

Objective: *Create a decorator that logs function metadata and parameters cleanly while preserving attributes.*

```
# Exercise 4: Execution Logger with Metadata Preservation
# Task: Create a decorator that logs function metadata and parameters cleanly.

from functools import wraps

def execution_logger(func):
    """Logs the entry, exit, and parameters of the decorated function while preserving attributes."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"--- [LOG] Entering function: {func.__name__} ---")
        print(f"[LOG] Arguments: positional={args}, keyword={kwargs}")

        # Execute the function and capture output
        try:
            result = func(*args, **kwargs)
            print(f"[LOG] Exiting function: {func.__name__} -> Return Value: {result}")
            return result
        except Exception as e:
            print(f"[LOG] Function {func.__name__} raised exception: {e}")
            raise e
    return wrapper

# Sample usage:
@execution_logger
def compute_area(length, width):
    """Calculates the area of a rectangle."""
    return length * width

if __name__ == "__main__":
    print(f"Docstring: {compute_area.__doc__}")
    compute_area(5, 10)
```

Explanation:

This logging pattern demonstrates full envelope wrapping. By capturing variables and formatting logs on entry and exit, the wrapper creates clean execution metrics. Using try-except ensures exceptions are logged while preserving standard traceback bubbling via raise e. Importantly, @wraps(func) copies metadata like __doc__ and __name__.

Exercise 5 Solution: Performance Monitor Stacking

Objective: *Stack multiple decorators to measure call count and execution durations simultaneously.*

```python
# Exercise 5: Performance Monitor Stacking
# Task: Stack multiple decorators to measure call count and execution durations simultaneously.

import time
from functools import wraps

def count_calls(func):
    """Tracks the total number of times a function has been executed."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        print(f"[Count] '{func.__name__}' call count: {wrapper.calls}")
        return func(*args, **kwargs)
    wrapper.calls = 0  # Initialize stateful counter attribute
    return wrapper

def timer(func):
    """Measures and prints the execution duration of a function call."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.perf_counter()
        result = func(*args, **kwargs)
        end_time = time.perf_counter()
        duration = end_time - start_time
        print(f"[Timer] '{func.__name__}' execution time: {duration:.6f} seconds")
        return result
    return wrapper

# Stack decorators: timer on top, count_calls on bottom
# Evaluation resolves as: timer(count_calls(heavy_computation))
@timer
@count_calls
def heavy_computation(n):
    """Simulates a heavy mathematical calculation loop."""
    total = 0
    for i in range(n):
        total += i * i
    return total

if __name__ == "__main__":
    heavy_computation(100000)
    heavy_computation(500000)
```

Explanation:

Stacking order maps to function nesting: timer(count_calls(heavy_computation)). When decorated this way, invoking heavy_computation() executes timer_wrapper, which measures how long it takes for count_calls_wrapper to execute (which in turn increments the counter and executes the target). If we reversed

the stack, the counter wrapper would include the timer overhead inside its count boundary.

Exercise 6 Solution: Stateful Retry Decorator

Objective: *Write a configurable decorator factory that retries failing operations on exceptions.*

```python
# Exercise 6: Stateful Retry Decorator Factory
# Task: Write a configurable decorator factory that retries failing operations.

import time
from functools import wraps

def retry(max_attempts=3, delay_seconds=1):
    """A decorator factory that retries execution on exception failures up to max_attempts."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            attempts = 0
            while attempts < max_attempts:
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    attempts += 1
                    print(f"[Retry] Attempt {attempts}/{max_attempts} failed with error: {e}")
                    if attempts >= max_attempts:
                        raise e
                    print(f"[Retry] Sleeping for {delay_seconds}s before next attempt...")
                    time.sleep(delay_seconds)
        return wrapper
    return decorator

# Sample usage:
@retry(max_attempts=3, delay_seconds=0.5)
def unstable_network_call(url):
    """Simulates an unstable HTTP endpoint call."""
    import random
    if random.random() > 0.3:
        raise ConnectionError("Timeout attempting to resolve endpoint.")
    return f"Response received from {url}!"

if __name__ == "__main__":
    try:
        msg = unstable_network_call("https://api.python-decorators.academy")
        print(f"Success Message: {msg}")
    except ConnectionError as e:
        print(f"Operation aborted after all retries: {e}")
```

Explanation:

This three-tier architecture is a standard pattern for configurable decorator factories. The top tier, retry(), accepts configurations like max_attempts, returning the standard decorator closure. Python then invokes this inner decorator, passing the target function object, which finally returns the wrapper enclosing both configurations and callables.

Exercise 7 Solution: Sliding Window Rate Limiter

Objective: *Build a stateful, rate-limiting decorator factory using closure states to monitor call frequencies.*

```python
# Exercise 7: Token Bucket Rate Limiter
# Task: Build a stateful, rate-limiting decorator factory using closure states.

import time
from functools import wraps

def limit_rate(max_calls=5, period_seconds=2):
    """Decorator factory that limits calls to a function within a specific time window."""
    def decorator(func):
        # Stateful closure boundary: timestamps list persists inside outer scope
        call_timestamps = []

        @wraps(func)
        def wrapper(*args, **kwargs):
            now = time.time()

            # Filter out timestamps outside our active sliding window
            # Use non-local list mutations to filter expired timestamps
            expired_threshold = now - period_seconds
            call_timestamps[:] = [t for t in call_timestamps if t > expired_threshold]

            if len(call_timestamps) >= max_calls:
                raise RuntimeError(
                    f"Rate Limit Exceeded: Function '{func.__name__}' is restricted to "
                    f"{max_calls} invocations every {period_seconds} seconds."
                )

            # Record current invocation timestamp and execute
            call_timestamps.append(now)
            return func(*args, **kwargs)
        return wrapper
    return decorator

# Sample usage:
@limit_rate(max_calls=3, period_seconds=2)
def read_secure_vault():
    """Simulates querying a secure storage system."""
    return "Decrypted Auth Token: s3cr3t_v@ult"

if __name__ == "__main__":
    for i in range(5):
        try:
            print(f"Call {i+1}: {read_secure_vault()}")
            time.sleep(0.3)
        except RuntimeError as e:
            print(f"Blocked call {i+1}: {e}")
```

Explanation:

By capturing a mutable list (call_timestamps) inside the decorator's enclosing scope, we hide stateful memory safe from external manipulation. The call_timestamps[:] slice assignment alters the original list in-place. If the window has more timestamps than max_calls, the wrapper blocks execution and raises an error, safeguarding APIs or resource-intensive calculations.

Exercise 8 Solution: Advanced Memoizer with Custom Cache

Objective: *Build an advanced memoization decorator that supports a shared, custom caching dictionary.*

```python
# Exercise 8: Configurable Memoization Cache
# Task: Build an advanced memoization decorator that supports a shared, custom caching dictionary.

from functools import wraps

def memoize(custom_cache_store=None):
    """Advanced memoization decorator factory that supports an optional custom storage container."""
    def decorator(func):
        # Create a default enclosed dict if no custom cache is provided
        cache = custom_cache_store if custom_cache_store is not None else {}

        @wraps(func)
        def wrapper(*args, **kwargs):
            # Create a serializable and hashable cache key from positional and keyword args
            # Sorting kwargs ensures identical keyword configurations yield matching keys
            key_args = args
            key_kwargs = tuple(sorted(kwargs.items()))
            cache_key = (key_args, key_kwargs)

            if cache_key in cache:
                print(f"[Memoize] Cache HIT for key: {cache_key}")
                return cache[cache_key]

            print(f"[Memoize] Cache MISS for key: {cache_key}. Computing...")
            result = func(*args, **kwargs)
            cache[cache_key] = result
            return result

        # Expose the active cache store object as a function attribute for debugging
        wrapper.cache_store = cache
        return wrapper
    return decorator

# Sample usage:
shared_database = {}

@memoize(custom_cache_store=shared_database)
def fibonacci(n):
    """Computes the nth Fibonacci number recursively with memoization."""
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

if __name__ == "__main__":
    print(f"Result (Fib 10): {fibonacci(10)}")
    print(f"Total Cached Keys in Database: {len(shared_database)}")
```

Explanation:

This memoizer creates a hashable cache key from both positional and keyword arguments. By checking custom_cache_store, it allows injecting an external shared dictionary. This enables advanced architecture designs like crossing execution boundaries or sharing cache pools across different decorated targets. The wrapper.cache_store = cache assignment exposes the private dictionary as a callable attribute for real-time monitoring and inspection.